

EEL 4742C: Embedded Systems

Name: Ryan Ramdihal

Lab 3: Timer Module

Introduction:

In this lab, the focus is on programming the MSP4306989's Timer_A module, in both continuous and up modes. The timer module operates by counting cycles based on a specified frequency, with the maximum count 65536 cycles owing to its 16-bit register. In continuous mode, the timer restarts upon reaching its maximum count, signifying the TAIFG being raised. The initial task involves configuring the timer to turn on the red LED whenever the flag is set, and turn off when the flag is lowered/reset, with the duration determined by the cycles, frequency and the divider. Additionally, the timer will be utilized in up mode to achieve a one-second timer period, equating to 32768 cycles for a 32kHz crystal. Finally, both continuous and up modes will be used to create a signal repeater. Where the microcontroller mimics the duration of the button being held using the red LED. provided that the duration does not exceed 64k cycle limit. If surpassed, the green LED will turn on and the S2 button must be pressed to reset the repeater. These exercises aim to provide a comprehensive understanding of the Timer_A's functions and its applications.

Part 3.1:

In this part of the lab, the objective is to flash the red LED using Timer_A in continuous mode via polling. First, configuring the ACLK to utilize a 32 KHz crystal by calling the function given, ensuring a stable time base for timer operations. Then configuring Timer_A to operate in continuous mode, with no divider, and clear its register ensuring it starts from zero. Inside the polling loop, the program continuously monitors the Timer_A interrupt flag (TAIFG) using a while loop. This loop effectively pauses execution until the flag is raised, indicating that Timer_A has reached its maximum count and reset. Upon detecting the flag, the program toggles the state of the red LED, turning it on or off. Finally, the interrupt flag (TAIFG) is cleared to prepare Timer_A for the next cycle. This approach of polling the Timer_A interrupt flag allows for synchronization of LED flashing with the timer's continuous count, achieving the desired flashing effect. The analysis of the delays were conducted below, shown in the table, and matched when tested.

0:IDx - Div by 1	$(65536)/32768 = 2\text{sec}$
1:IDx - Div by 2	$(65536*2)/32768 = 4\text{sec}$
2:IDx - Div by 4	$(65536*4)/32768 = 8\text{sec}$
3:IDx - Div by 8	$(65536*8)/32768 = 16\text{sec}$

3.1 Code:

```
// Flashing the LED with Timer_A, continuous mode, via polling
#include <msp430fr6989.h>
#define redLED BIT0 // Red LED at P1.0
```

```

#define greenLED BIT7 // Green LED at P9.7

// Configures ACLK to 32 KHz crystal
void config_ACLK_to_32KHz_crystal() {

    // By default, ACLK runs on LFMODCLK at 5MHz/128 = 39 KHz
    // Reroute pins to LFXIN/LFXOUT functionality
    PJSEL1 &= ~BIT4;
    PJSEL0 |= BIT4;

    // Wait until the oscillator fault flags remain cleared
    CSCTL0 = CSKEY; // Unlock CS registers
    do {
        CSCTL5 &= ~LFXTOFFG; // Local fault flag
        SFRIFG1 &= ~OFIFG; // Global fault flag
    } while((CSCTL5 & LFXTOFFG) != 0); //clears fault flags, then crystal clock ready to use

    CSCTL0_H = 0; // Lock CS registers
    return;
}

void main(void) {
    // Stop the Watchdog timer
    WDTCTL = WDTPW | WDTHOLD;

    // Unlock the GPIO pins
    PM5CTL0 &= ~LOCKLPM5;

    // Configure the LEDs as output
    P1DIR |= redLED;
    P9DIR |= greenLED;
//TA0CTL is timer0_A
    // Configure ACLK to the 32 KHz crystal (function call)
    config_ACLK_to_32KHz_crystal();
    // Configure Timer_A //counts from 0 to 2^16
    // Use ACLK, divide by 1, continuous mode, clear TAR
    TA0CTL = TASSEL_1 | ID_0 | MC_2 | TACLRL; //TACLRL to make sure its clear before starting
    //MC runs the timer if set to 1 or 2
    // Ensure flag is cleared at the start
    TA0CTL &= ~TAIFG;

    // Infinite loop
    for(;;) {
        // Wait in this empty loop for the flag to raise
        while( !(TA0CTL & TAIFG)) {} //while flag is not raised

        // Do the action here
        // Toggle the red LED
        P1OUT ^= redLED; //when raised turn on led
        // Clear the interrupt flag
        TA0CTL &= ~TAIFG; //to reset flag 0 to turn off led
    }
}

```

Part 3.2:

The objective is to utilize Timer_A in up mode to achieve a one-second timer period, resulting in the red LED flashing at one-second intervals. This is achieved by first configuring ACLK to utilize a 32 KHz crystal oscillator, ensuring a stable time base. The value of TA0CCR0 is set to 32768, as this value corresponds to one second when ACLK is configured to run at 32 KHz. Next Timer_A is configured to up mode and with no divider. The timer generates an interrupt when it reaches this value, effectively creating a one-second timer period. Inside the polling loop, the program waits for the TAIFG to be raised, indicating that the one-second interval has elapsed. Once the flag is raised, the program toggles the state of the red LED and clears the interrupt flag (TAIFG) to prepare Timer_A for the next cycle. This approach ensures that the red LED flashes at one-second intervals, demonstrating the accurate timing capabilities of Timer_A in up mode. The value that TA0CCR0 needs to be to achieve 0.1 and 0.01 seconds are shown below in the table.

$32768 * 1/32768 =$	1 sec
$3276.8 * 1/32768 =$	0.1 sec
$327.68 * 1/32768 =$	0.01 sec

3.2 Code:

```
#include <msp430fr6989.h>
#define redLED BIT0 // Red LED at P1.0
#define greenLED BIT7 // Green LED at P9.7

// Configures ACLK to 32 KHz crystal
void config_ACLK_to_32KHz_crystal() {
    // By default, ACLK runs on LFMODCLK at 5MHz/128 = 39 KHz
    // Reroute pins to LFXIN/LFXOUT functionality
    PJSEL1 &= ~BIT4;
    PJSEL0 |= BIT4;
    // Wait until the oscillator fault flags remain cleared
    CSCTL0 = CSKEY; // Unlock CS registers
    do {
        CSCTL5 &= ~LFXTOFFG; // Local fault flag
        SFRIFG1 &= ~OFIFG; // Global fault flag

    } while((CSCTL5 & LFXTOFFG) != 0);
    CSCTL0_H = 0; // Lock CS registers
    return;
}

void main(void) {
    // Stop the Watchdog timer
    WDTCTL = WDTPW | WDTHOLD;

    // Unlock the GPIO pins
    PM5CTL0 &= ~LOCKLPM5;
```

```

// Configure the LEDs as output
P1DIR |= redLED;
P9DIR |= greenLED;
// Configure ACLK to the 32 KHz crystal (function call)

config_ACLK_to_32KHz_crystal();
TA0CCR0 = 32768;
// 32768 *1/32768 = 1sec, 3276.8 *1/32768 = 0.1 sec, 327.68 *1/32768 = 0.01 sec, at this visibly //looks fully toggled
//on
// Configure Timer_A
// Use ACLK, divide by 1, up mode, clear TAR
TA0CTL = TASSEL_1 | ID_0 | MC_1 | TACLK;
// Ensure flag is cleared at the start
TA0CTL &= ~TAIFG;
// Infinite loop
for(;;) {
    // Wait in this empty loop for the flag to raise
    while( !(TA0CTL & TAIFG)) {}
    // Do the action here
    P1OUT ^= redLED; // Toggle the red LED
    TA0CTL &= ~TAIFG; // Clear the interrupt flag
}
}

```

Part 3.3:

Lastly, the repeater is implemented using both continuous and up modes of Timer_A. The ACLK is configured to utilize a 32 KHz crystal oscillator, like the previous parts. The approach involves utilizing Timer_A in both continuous and up modes to measure the duration of a button press and replicate it using the red LED. Initially, Timer_A is configured in continuous mode (MC_0), and its count value (TA0CCR0) is set to zero. When button 1 (BUT1) is pressed, Timer_A starts counting continuously. The program waits until either button 1 is released or Timer_A raises its interrupt flag (TAIFG). If the interrupt flag is raised, indicating that the button press duration exceeds the maximum count limit, the green LED is turned on to indicate an error. The program waits until button 2 (BUT2) is pressed to reset the repeater, and then Timer_A is stopped and cleared. If the button press duration is within limits, Timer_A's count value is copied to TA0CCR0, and Timer_A is stopped and cleared for another input. In the second part of the loop, if TA0CCR0 has a non-zero value (indicating a valid button press duration) and Timer_A's interrupt flag is not raised, Timer_A is configured in up mode (MC_1). The red LED is turned on, and the program waits until Timer_A raises its interrupt flag, indicating that the duration specified by TA0CCR0 has elapsed. Once the flag is raised, the red LED is turned off, Timer_A is stopped and cleared, and TA0CCR0 is reset to zero, preparing Timer_A for the next button press. This approach allows the microcontroller to accurately replicate button press durations using the red LED while handling error conditions and ensuring proper synchronization with Timer_A. The tradeoff between dividers is that it increases the delay as shown in the table below.

Max Delay Equation	Seconds
--------------------	---------

$(65536 \cdot 1) / 32768$	2 seconds
$(65536 \cdot 2) / 32768$	4 seconds
$(65536 \cdot 4) / 32768$	8 seconds
$(65536 \cdot 8) / 32768$	16 seconds

3.3 Code:

```
#include <msp430fr6989.h>
#include <stdint.h>
#define redLED BIT0 // Red LED at P1.0
#define greenLED BIT7 // Green LED at P9.7
#define BUT1 BIT1 // Button S1 at P1.1
#define BUT2 BIT2 //P1.2
// Configures ACLK to 32 KHz crystal
void config_ACLK_to_32KHz_crystal() {
// By default, ACLK runs on LFMODCLK at 5MHz/128 = 39 KHz
// Reroute pins to LFXIN/LFXOUT functionality
PJSEL1 &= ~BIT4;
PJSEL0 |= BIT4;
// Wait until the oscillator fault flags remain cleared
CSCTL0 = CSKEY; // Unlock CS registers
do {
CSCTL5 &= ~LFXTOFFG; // Local fault flag
SFRIFG1 &= ~OFIFG; // Global fault flag
} while((CSCTL5 & LFXTOFFG) != 0);
CSCTL0_H = 0; // Lock CS registers
return;
}
void main(void) {
WDTCTL = WDTPW | WDTHOLD; // Stop the Watchdog timer
PM5CTL0 &= ~LOCKLPM5; // Enable the GPIO pins
// Configure and initialize LEDs
P1DIR |= redLED; // Direct pin as output
P9DIR |= greenLED; // Direct pin as output
P1OUT &= ~redLED; // Turn LED Off
P9OUT &= ~greenLED; // Turn LED Off
// Configure buttons
P1DIR &= ~(BUT1 | BUT2); // Direct Pin as input, uses both buttons
P1REN |= (BIT1 | BUT2); // Enable built-in resistor for both buttons
P1OUT |= (BIT1 | BUT2); // Set resistor as pull-up for both buttons
// Configure ACLK to the 32 KHz crystal (function call)
config_ACLK_to_32KHz_crystal();
//Configure CCR0 value
//TA0CCR0 = 32768; //32 * 2^10 -> K => 2^10
// Configure Timer_A
// Use ACLK, divide by 1, continuous mode, clear TAR
TA0CTL = TASSEL_1 | ID_0 | MC_0 | TACLK;
// Ensure flag is cleared at the start
TA0CTL &= ~TAIFG;
```

```

// Infinite loop
for(;;) {
    TA0CCR0 = 0; //Ensure TA0CCR0 is set to a 0 state
    if ( ((P1IN & BUT1) == 0) ) { //If button 1 has been pressed
        TA0CTL |= MC_2; //Start continuous timer
        while ( ((P1IN & BUT1) == 0) && ((TA0CTL & TAIFG) != TAIFG) ) {} //While button is pressed and flag is not up
        TA0CTL &= ~MC_3; //Stop timer, keep TA0R
        if((TA0CTL & TAIFG) == TAIFG){ //If continuous timer raised flag
            P9OUT |= greenLED; //Turn on error-flag LED
            while ((P1IN & BUT2) != 0){} //While button 2 is not pressed
            P9OUT &= ~greenLED; //Turn off green LED
            TA0CTL &= ~(TAIFG|MC_3); //Reset flag,stop timer
            TA0CTL |= TACLRL; //Clear TAR
            continue; //Exit parent if statement without running remaining code
        }
        TA0CCR0 = TA0R; //Copy timer to TA0CCR0,max delay when flag is not raised
        TA0CTL |= TACLRL; //Clear TAR
        TA0CTL &= ~MC_3;
    }
    if(( TA0CCR0 != 0) && (TA0CTL & TAIFG) != TAIFG) { //IF TA0CCR0 has been altered
        TA0CTL |= MC_1 | TACLRL; //Start timer in up-mode & ensure timer is cleared
        P1OUT |= redLED; //Turn on LED
        while((TA0CTL & TAIFG) != TAIFG) {} //Wait until flag is raised
        P1OUT &= ~redLED; //Turn off LED
        TA0CTL &= ~(TAIFG | MC_3); //Clear Flag,stop timer
        TA0CCR0 = 0; //Clear TA0CCR0
        TA0CTL |= TACLRL; //Clear TAR
    }
}
}
}

```

Conclusion:

In conclusion, this lab provided a comprehensive understanding toward the MSP430FR6989's Timer_A module in both continuous and up modes, to achieve timing functionalities. In part 1, Timer_A operated in continuous mode to flash the red LED at intervals determined by the timer's maximum count value. Subsequently, in part two, Timer_A was configured in up mode to create a one-second timer period, demonstrating its versatility in different timing scenarios. Finally, part three integrated both continuous and up modes to construct a signal repeater, replicating button press durations using the red LED while handling error conditions effectively. Through these exercises, a thorough understanding of Timer_A's capabilities and its application in real-time systems was achieved, highlighting its significance in embedded systems development and time-sensitive applications.

Student Q&A

1. So far, we have seen two ways of timing delays: using a delay loop and using Timer A. Which approach provides more control and accuracy over the delays? Explain.

Timer_A provides more control and accuracy over the delays. This is because the delays can be controlled by the timer's parameter values outside of the CPU, allowing for accurate time delays, whereas delay loops depend on the CPU speed.

2. Explain the polling technique and how it's used in this lab.

The polling technique continuously checks the status of the hardware to determine when a condition is met. In the context of this lab, the polling technique is used to synchronize the LED blinking with the timer. The program continuously checks the flag status, on whether it is raised or not, indicating if the timer reached its maximum count.

3. Is the polling technique a suitable choice when we care about saving battery power? Explain.

No, polling is not a suitable choice when it comes to saving battery power. The CPU stays active, which in turn consumes more power. Interrupts are the solution instead of this technique.

4. If we write 0 to TAR using a line code, does TAIFG go to 1?

No, writing 0 to TAR does not raise the TAIFG. The TAIFG will only be set to 1 when the Timer_A counter reaches over its maximum value.

5. From what we have seen in this lab, which mode gives us more control over the timing duration: the up mode or the continuous mode?

The up mode provides more control over time duration. In up mode, we can determine the duration of the time period, making it suitable for precision timing.

6. In this lab, you were given a summary of the timer's main control register, TACTL, and the fields within. To practice reading the documentation, find this information in the Family User's Guide (slau367o) document and include a screenshot of TACTL's layout and the table describing the fields within it. This information can be found at the end of the Timer A chapter in the Family User's Guide.

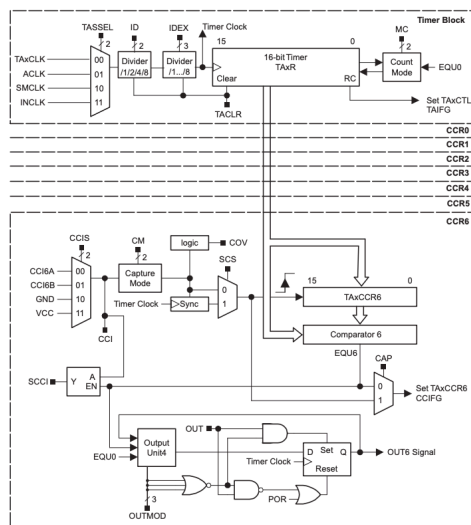


Figure 25-1: Timer_A Block Diagram

25.3.1 TAxCTL Register

Timer_Ax Control Register

Figure 25-16. TAxCTL Register

15	14	13	12	11	10	9	8
Reserved			Reserved			TASSEL	
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)
7	6	5	4	3	2	1	0
ID		MC		Reserved	TACLR	TAIE	TAIFG
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	w-(0)	rw-(0)	rw-(0)

Table 25-4. TAxCTL Register Description

Bit	Field	Type	Reset	Description
15-10	Reserved	RW	0h	Reserved
9-8	TASSEL	RW	0h	Timer_A clock source select 00b = TAxCLK 01b = ACLK 10b = SMCLK 11b = INCLK
7-6	ID	RW	0h	Input divider. These bits along with the TAIDEX bits select the divider for the input clock. 00b = /1 01b = /2 10b = /4 11b = /8
5-4	MC	RW	0h	Mode control. Setting MC = 00h when Timer_A is not in use conserves power. 00b = Stop mode: Timer is halted 01b = Up mode: Timer counts up to TAxCCR0 10b = Continuous mode: Timer counts up to 0FFFFh 11b = Up/down mode: Timer counts up to TAxCCR0 then down to 0000h
3	Reserved	RW	0h	Reserved
2	TACLR	RW	0h	Timer_A clear. Setting this bit clears TAR, the clock divider logic (the divider setting remains unchanged), and the count direction. The TACLR bit is automatically reset and is always read as zero.
1	TAIE	RW	0h	Timer_A interrupt enable. This bit enables the TAIFG interrupt request. 0b = Interrupt disabled 1b = Interrupt enabled
0	TAIFG	RW	0h	Timer_A interrupt flag 0b = No interrupt pending 1b = Interrupt pending